Python Engineering - Concept Guide

Class 09 - Modules & Imports: main.py, process_trajectory(), and __name__

Step-by-step guide to finish EX17. The goal is not to change the physics, but to separate responsibilities: each module computes its part and main.py coordinates the program and presents results.

1. Mental architecture: three files, three responsibilities

File	Responsibility	What it should NOT do
vertical_motion.py	Physics, Euler, simulation, analytical solution, errors	Configure the case or present final report
data_processing.py	Masks, filters, statistics, energy, processed dataset	Run simulation or choose configuration
main.py	Configure, call functions, present results	Contain Euler equations or filtering details

Simple rule: vertical_motion computes the flight; data_processing analyzes the data; main decides what to execute and display.

2. Complete flow

```
main.py
  |-- config
  |-- validate_inputs(config)
  |-- simulate_vertical_motion(config)
  |     `--> results, trajectory, trajectory_array
  |-- analytical_solution(config)
  |-- calculate_errors(...)
  |-- process_trajectory(trajectory_array, config)
  |     `--> processing_results, processed_data
  `-- print everything
```

3. What is process_trajectory()?

It receives simulation data and turns it into useful information. It does not recompute the trajectory; it only processes it.

Element	Content
Input 1	trajectory_array: [h, v, t] matrix
Input 2	config: dictionary containing g, dt, etc.
Output 1	processing_results: dictionary with summaries, filters, indices, statistics
Output 2	processed_data: [h, v, t, energy] matrix

4. data_processing.py - recommended skeleton

```
import numpy as np

def process_trajectory(trajectory_array, config):
    heights = trajectory_array[:, 0]
    velocities = trajectory_array[:, 1]
    times = trajectory_array[:, 2]
    ascending_mask = velocities > 0
    descending_mask = velocities < 0
    above_100_mask = heights > 100
    time_window_mask = (times >= 2.0) & (times <= 5.0)
    high_ascending_mask = (heights > 50) & (velocities > 0)
    energy = config['g'] * heights + 0.5 * velocities**2
    processed_data = np.column_stack((trajectory_array, energy))
    dt_samples = np.diff(times)
    processing_results = {
        'n_ascending': np.sum(ascending_mask),
        'n_descending': np.sum(descending_mask),
        'states_above_100': trajectory_array[above_100_mask],
        'states_time_2_5': trajectory_array[time_window_mask],
        'states_high_ascending': trajectory_array[high_ascending_mask],
        'mean_height': np.mean(heights),
        'std_height': np.std(heights),
        'mean_velocity': np.mean(velocities),
        'std_velocity': np.std(velocities),
        'indices_above_100': np.where(above_100_mask)[0],
        'dt_samples': dt_samples,
        'dt_constant': np.allclose(dt_samples, config['dt'])
    }
    return processing_results, processed_data
```

The function does not print. Its job ends when it returns data.

5. Why return a dictionary?

Because there are many results. A dictionary lets us access them by name instead of remembering positions.

```
processing_results['mean_height']
processing_results['dt_constant']
processing_results['states_above_100']
```

6. What stays in main.py?

main.py acts like an orchestra conductor: it decides when each component runs.

```
from vertical_motion import (
    validate_inputs, simulate_vertical_motion,
    analytical_solution, calculate_errors
)
from data_processing import process_trajectory

def main():
    config = {'h0':0.0, 'v0':50.0, 't0':0.0, 'g':9.81, 'dt':0.1}
    if validate_inputs(config):
        results, trajectory, trajectory_array = \
            simulate_vertical_motion(config)
        analytical_results = analytical_solution(config)
        differences, errors = calculate_errors(...)
        processing_results, processed_data = \
            process_trajectory(trajectory_array, config)
        # Presentation only from here
        print(results)
        print(analytical_results)
        print(errors)
        print(processing_results)

if __name__ == '__main__':
    main()
```

7. So yes: prints belong in main

Your intuition is correct. In a clean architecture, functions compute and return; main.py chooses how to present the information.

For EX17 you do not need to remove every existing print inside simulate_vertical_motion() immediately. The professional direction is **calculation separated from presentation**.

8. __name__ explained simply

Python automatically creates a special variable named __name__ inside every file.

Case A - execute a file directly:

```
python main.py

# Inside main.py:
__name__ == '__main__'
```

Case B - import a file:

```
import vertical_motion

# Inside vertical_motion.py:
  name   == 'vertical_motion'
```

The condition literally asks: **"Am I the file the user executed directly?"**

9. What exactly does the main guard do?

```
if __name__ == '__main__':
    main()
```

When main.py is executed directly, the condition is True and main() runs. If another file imports main, the condition is False: definitions are loaded, but the simulation does not start automatically.

Analogy: a file can be both a machine and a toolbox. If switched on directly, start the machine; if imported, expose the tools.

10. Why define def main()?

Python does not require it, but it organizes the program flow, avoids uncontrolled top-level execution, and helps testing/reuse.

11. Imports: three forms

Syntax	What it looks for	What becomes available	Example
import module	a module	the module	vertical_motion.simulate_vertical_motion(...)
import module as alias	a module	an alias	np.array(...)
from module import func	a specific name	the function	simulate_vertical_motion(...)

12. Each module imports what it uses

Imports are not magically shared between files. If data_processing.py uses NumPy, it must contain `import numpy as np`.

```
# data_processing.py
import numpy as np

# vertical_motion.py
from math import sqrt
import numpy as np

# main.py
# No sqrt or np if main does not use them directly.
```

After redistribution, remove unused external imports from main.py.

13. What I would NOT move yet

Keep analytical_solution() and calculate_errors() in vertical_motion.py. They belong to numerical/physical simulator validation. Splitting them further now would fragment EX17 too much.

14. EX17 completion checklist

```
[ ] Create data_processing.py
[ ] Add import numpy as np inside it
[ ] Create process_trajectory(trajectory_array, config)
[ ] Move masks/statistics/energy/dt processing into it
[ ] Return processing_results, processed_data
[ ] Import process_trajectory in main.py
[ ] Call analytical_solution() and calculate_errors() from main.py
[ ] Call process_trajectory() from main.py
[ ] Leave final presentation/prints in main.py
[ ] Remove unused imports from main.py
[ ] Run python main.py
[ ] Verify results match EX16
```

15. Final EX17 architecture

```
ex17_vertical_motion_modules/
|-- main.py
|   config / orchestration / prints
|-- vertical_motion.py
|   validation / Euler / simulation / analytical / errors
`-- data_processing.py
    masks / filtering / statistics / energy / processed dataset
```